

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

По дисциплине “Системное программирование”

Специальность “Программная инженерия”

Выполнил:

Я.В. Автушенко

Студент группы ПО-13

Проверил:

А.Д. Кулик

преп.-стажёр кафедры ИИТ

Цель: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python

Задание 1:

Первая группа заданий (порождающий паттерн)

1) Кофе-автомат с возможностью создания различных кофейных напитков (предусмотреть 5 классов наименований)

```
from abc import ABC, abstractmethod
```

```
# Абстрактный класс напитка
```

```
class Coffee(ABC):
```

```
    @abstractmethod
```

```
    def get_description(self): pass
```

```
    @abstractmethod
```

```
    def get_cost(self): pass
```

```
    def __str__(self): return f"{self.get_description(): {self.get_cost()} руб."
```

```
# Конкретные напитки
```

```
class Espresso(Coffee):
```

```
    def get_description(self): return "Эспрессо"
```

```
    def get_cost(self): return 100
```

```
class Americano(Coffee):
```

```
    def get_description(self): return "Американо"
```

```
    def get_cost(self): return 120
```

```
class Cappuccino(Coffee):
```

```
    def get_description(self): return "Капучино"
```

```
    def get_cost(self): return 150
```

```
class Latte(Coffee):
```

```
    def get_description(self): return "Латте"
```

```
    def get_cost(self): return 160
```

```
class Mocha(Coffee):
```

```
    def get_description(self): return "Мокко"
```

```
    def get_cost(self): return 180
```

```
# Фабрика (Кофе-автомат)
```

```
class CoffeeMachine:
```

```
    def __init__(self):
```

```
        self._recipes = {
```

```
            "эспрессо": Espresso,
```

```
            "американо": Americano,
```

```
            "капучино": Cappuccino,
```

```
            "латте": Latte,
```

```
            "мокко": Mocha
```

```
        }
```

```
    def make_coffee(self, name: str) -> Coffee:
```

```

        name = name.lower()
        if name not in self._recipes:
            raise ValueError(f'Напиток '{name}' не найден. Доступны: {',
'.join(self._recipes.keys())}')
        return self._recipes[name]()

    def show_menu(self):
        print("=== МЕНЮ ===")
        for name, coffee_class in self._recipes.items():
            coffee = coffee_class()
            print(f" {coffee}")
        print("=====")

# Демонстрация
if __name__ == "__main__":
    machine = CoffeeMachine()
    machine.show_menu()

    print("\nЗаказы:")
    for drink in ["эспрессо", "капучино", "латте", "мокко"]:
        coffee = machine.make_coffee(drink)
        print(f" Приготовлен: {coffee}")

    # Попытка заказать несуществующий напиток
    try:
        machine.make_coffee("чай")
    except ValueError as e:
        print(f"\nОшибка: {e}")

```

```

=== МЕНЮ ===
Эспрессо: 100 руб.
Американо: 120 руб.
Капучино: 150 руб.
Латте: 160 руб.
Мокко: 180 руб.
=====

Заказы:
Приготовлен: Эспрессо: 100 руб.
Приготовлен: Капучино: 150 руб.
Приготовлен: Латте: 160 руб.
Приготовлен: Мокко: 180 руб.

Ошибка: Напиток 'чай' не найден. Доступны: эспрессо, американо, капучино, латте, мокко
Для продолжения нажмите любую клавишу . . .

```

Задание 2:

Вторая группа заданий (структурный паттерн)

1) Проект «Часы». В проекте должен быть реализован класс, который дает возможность пользоваться часами со стрелками так же, как и цифровыми часами. В классе «Часы со стрелками» хранятся повороты стрелок.

```
from abc import ABC, abstractmethod
```

```
# Целевой интерфейс (цифровые часы)
```

```
class DigitalClock(ABC):
```

```
    @abstractmethod
```

```
    def get_time(self) -> str: pass
```

```
# Адаптируемый класс (часы со стрелками)
```

```
class AnalogClock:
```

```
    def __init__(self, hours: int = 0, minutes: int = 0):
```

```
        self.hours = hours % 12
```

```
        self.minutes = minutes % 60
```

```
    def get_hours_angle(self) -> float:
```

```
        return self.hours * 30 + self.minutes * 0.5
```

```
    def get_minutes_angle(self) -> float:
```

```
        return self.minutes * 6
```

```
    def set_time(self, hours: int, minutes: int):
```

```
        self.hours = hours % 12
```

```
        self.minutes = minutes % 60
```

```
    def __str__(self):
```

```
        return f"Стрелки:  
минуты={self.get_minutes_angle():.1f}°"
```

```
        часы={self.get_hours_angle():.1f}°,
```

```
# Адаптер (часы со стрелками -> цифровые)
```

```
class AnalogToDigitalAdapter(DigitalClock):
```

```
    def __init__(self, analog_clock: AnalogClock):
```

```
        self._clock = analog_clock
```

```
def get_time(self) -> str:
    h, m = self._clock.hours, self._clock.minutes
    return f'{h:02d}:{m:02d}'

def set_time(self, hours: int, minutes: int):
    self._clock.set_time(hours, minutes)

def __str__(self):
    return f"Цифровые: {self.get_time()} | {self._clock}"
```

Демонстрация

```
if __name__ == "__main__":
    # Часы со стрелками
    analog = AnalogClock(3, 45)
    print("Только аналоговые:")
    print(f" {analog}")

    # Адаптер для использования как цифровых
    adapter = AnalogToDigitalAdapter(analog)
    print("\nЧерез адаптер:")
    print(f" {adapter}")
    print(f" Время: {adapter.get_time()}")

    # Изменение времени
    print("\nУстановка 10:30:")
    adapter.set_time(10, 30)
    print(f" {adapter}")

    print("\nУстановка 0:15:")
    adapter.set_time(0, 15)
    print(f" {adapter}")
```

```
Только аналоговые:
  Стрелки: часы=112.5°, минуты=270.0°

Через адаптер:
  Цифровые: 03:45 | Стрелки: часы=112.5°, минуты=270.0°
  Время: 03:45

Установка 10:30:
  Цифровые: 10:30 | Стрелки: часы=315.0°, минуты=180.0°

Установка 0:15:
  Цифровые: 00:15 | Стрелки: часы=7.5°, минуты=90.0°
Для продолжения нажмите любую клавишу . . .
```

Задание 3:

Третья группа заданий (поведенческий паттерн)

1) Проект «Клавиатура настраиваемого калькулятора». Цифровые и арифметические кнопки имеют фиксированную функцию, а остальные могут менять своё назначение.

```
from abc import ABC, abstractmethod
```

```
# Команда
```

```
class Command(ABC):
```

```
    @abstractmethod
```

```
    def execute(self, a: float, b: float) -> float: pass
```

```
    @abstractmethod
```

```
    def get_symbol(self) -> str: pass
```

```
# Конкретные команды
```

```
class Add(Command):
```

```
    def execute(self, a, b): return a + b
```

```
    def get_symbol(self): return "+"
```

```
class Subtract(Command):
```

```
    def execute(self, a, b): return a - b
```

```
    def get_symbol(self): return "-"
```

```
class Multiply(Command):  
    def execute(self, a, b): return a * b  
    def get_symbol(self): return "×"
```

```
class Divide(Command):  
    def execute(self, a, b):  
        if b == 0: raise ValueError("Деление на ноль")  
        return a / b  
    def get_symbol(self): return "÷"
```

```
class Power(Command):  
    def execute(self, a, b): return a ** b  
    def get_symbol(self): return "^"
```

```
class Modulo(Command):  
    def execute(self, a, b): return a % b  
    def get_symbol(self): return "%"
```

Калькулятор с настраиваемыми кнопками

```
class Calculator:  
    def __init__(self):  
        self._custom_buttons = {}  
        self._operations = {  
            "+": Add(), "-": Subtract(), "×": Multiply(), "÷": Divide()  
        }  
  
    def assign_button(self, key: str, command: Command):  
        self._custom_buttons[key] = command  
  
    def press(self, key: str, a: float, b: float) -> float:  
        # Фиксированные кнопки
```

```

    if key in self._operations:
        return self._operations[key].execute(a, b)

# Настраиваемые кнопки
    if key in self._custom_buttons:
        return self._custom_buttons[key].execute(a, b)
    raise ValueError(f"Кнопка '{key}' не назначена")

def show_buttons(self):
    print("Фиксированные:", " ".join(self._operations.keys()))
    print("Настраиваемые:",
          " ".join(f"{k}({v.get_symbol()})" for k, v in self._custom_buttons.items()))
    if self._custom_buttons else "нет")

# Демонстрация
if __name__ == "__main__":
    calc = Calculator()
    calc.show_buttons()

# Фиксированные операции
    print(f"\n5 + 3 = {calc.press('+', 5, 3)}")
    print(f"10 ÷ 4 = {calc.press('÷', 10, 4)}")

# Назначаем настраиваемые кнопки
    calc.assign_button("F1", Power())
    calc.assign_button("F2", Modulo())
    calc.show_buttons()

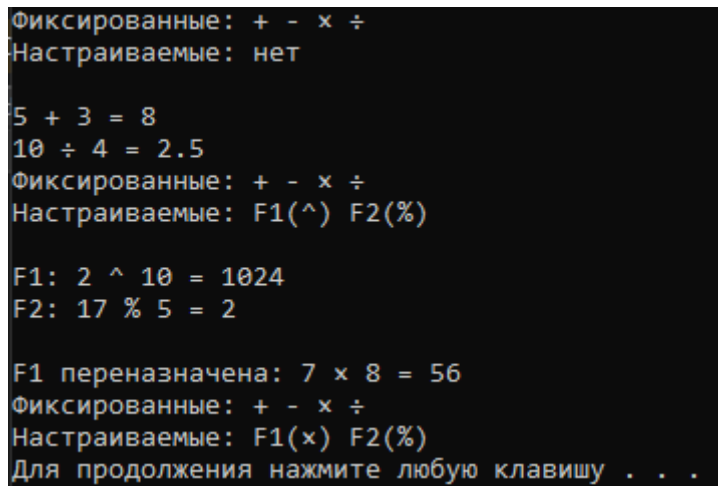
# Используем настраиваемые
    print(f"\nF1: 2 ^ 10 = {calc.press('F1', 2, 10)}")
    print(f"F2: 17 % 5 = {calc.press('F2', 17, 5)}")

# Переназначаем кнопку

```



```
calc.assign_button("F1", Multiply())  
print(f"\nF1 переназначена:  $7 \times 8 = {calc.press('F1', 7, 8)}$ ")  
calc.show_buttons()
```



```
Фиксированные: + - x ÷  
Настраиваемые: нет  
  
5 + 3 = 8  
10 ÷ 4 = 2.5  
Фиксированные: + - x ÷  
Настраиваемые: F1(^) F2(%)  
  
F1: 2 ^ 10 = 1024  
F2: 17 % 5 = 2  
  
F1 переназначена: 7 x 8 = 56  
Фиксированные: + - x ÷  
Настраиваемые: F1(x) F2(%)  
Для продолжения нажмите любую клавишу . . .
```

Вывод: были отработаны базовые знания языка программирования Python при решении практических задач